VEXON DEVELOPER HANDBOOK · GUIDE 05

VEXON Feature-by-Feature Guide

Every significant module: who uses it, which files implement it, how one workflow runs end to end, and what is tested

Written for	A student who needs to debug or change one VEXON module and wants to know exactly where to look
Project	VEXON / CampusConnect (repository: this folder's parent project)
Evidence	Generated from a direct inspection of the repository source code
Repository state	branch fix/csrf-legacy-cookie, commit af623fb, plus 126 uncommitted working-tree change(s)
Generated	16 October 2026, 21:11
Rebuild	python docs/developer-handbook/build_handbook.py

Accuracy rule: every statement about VEXON in this guide is based on files in the repository. Anything that could not be confirmed is labelled 'unverified', 'planned' or 'hypothetical'. No secrets, passwords or real student data are reproduced.

Contents

How to read this guide . . . 3
Module status table . . . 3
Authentication and account security . . . 4
Roles, users and role assignments . . . 5
Dashboard and analytics . . . 5
Announcements, clubs, events and discussions . . . 5
People directory, profiles and search . . . 6
Achievements . . . 6
Attendance (students) . . . 6
Faculty attendance, early exit and period cover . . . 7
Timetable, subjects, holidays and exams . . . 8
Study materials and the JNN Study Assistant (AI) . . . 8
Chat . . . 9
Notifications and push . . . 9
Gate pass . . . 9
Lost and found . . . 10
Complaints (grievances) . . . 10
SOS alerts . . . 11
Hostel . . . 11
Bus tracking . . . 11
Alumni network and mentorship . . . 12
Android: the Kotlin app . . . 12
Android: the Expo app (older) . . . 12
iOS app . . . 13
Practice: explain one workflow end to end . . . 13
Source references . . . 13

How to read this guide

Each module section answers the same ten questions: purpose, roles, frontend entry points, backend routes and logic, models, real-time and external services, validation and security, files to inspect when debugging, tests and gaps, and one workflow step by step. Paths are relative to the repository root. Web pages live under `client/src/pages/` and controllers under `server/src/controllers/`. "Tests" means files in `server/tests/` unless stated otherwise.

> **Evidence status**
> Test evidence. During this audit (16 October 2026) the server suite, client unit tests, Expo unit tests and Kotlin unit tests were **actually run**. Guide 07 records the results. "Covered by" below means a test file exercises that module's API. It does **not** mean every path is tested.

Module status table

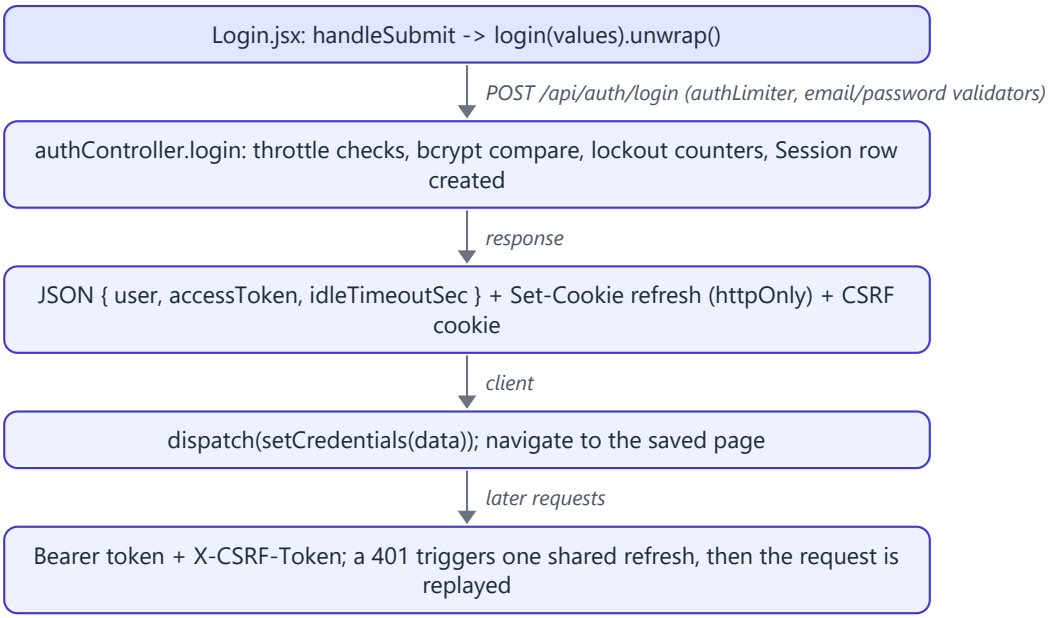
Module	Web	Kotlin app	Server tests	Status
Authentication, sessions, security logs	Yes	Yes	`auth`, `idle-session`, `csrf-*`, `refresh-rate-limit`, `security-*`, `accounts-security-logs`, `admin-login-restrictions`	Implemented and verified
Roles and role assignments	Yes	Partial (create user)	`roles`, `role-assignments`, `college-wide-roles`, `first-year-hod`, `class-in-charge*`	Implemented and verified
Dashboard and analytics	Yes	Home tiles	`hod-dashboard-cards`, `faculty-class-cards`, partial	Implemented, partly tested
Announcements	Yes	Yes	`announcement-audience`, `announcement-tampering`	Implemented and verified
Clubs	Yes	Yes	No direct API tests (one moderation test touches clubs)	Implemented, **not tested**
Events	Yes	Yes	`events-management`, `event-attendance-integration`	Implemented and verified
Discussions, reports, moderation	Yes	Yes	No discussion API tests; one report test	Implemented, **mostly untested**
People directory, profiles, search	Yes	Yes	`people`, `student-people`, `security-student-lists`	Implemented and verified
Achievements	Yes	Yes	`achievements`	Implemented and verified
Student attendance, corrections, reports	Yes	Yes	`attendance-*`, `od-leave-scope`, `exam-timetable`	Implemented and verified
Faculty attendance, early exit, cover	Yes	Partial	`faculty-cover`, `attendance-monitor`	Implemented and verified
Timetable, subjects, holidays, exams	Yes	Yes	`timetable-*`, `holidays`, `exam-timetable`, `academic*`	Implemented and verified
Study materials and AI assistant	Yes	Materials	`study-materials`, `study-assistant` (the Groq API is mocked or absent in tests)	Implemented; the live AI was **not verified**
Chat	Yes	Yes	`chat*` (6 files)	Implemented and verified
Notifications and push	Yes	Yes	`notifications-push`	In-app verified; real FCM delivery **not verified**
Gate pass (all variants)	Yes	Yes	`campus`, `gatepass-*`, `gate-*`, `emergency-gate-pass`, `day-scholar-*`, `outpass-otp`, `otp-*`, `parent-otp-*`	Implemented and verified; a real SMS send was **not verified**
Lost and found	Yes	Yes	3 tests in `campus.test.js`	Implemented, partly tested

Module	Web	Kotlin app	Server tests	Status
Complaints	Yes	Yes	`complaints`	Implemented and verified
SOS	Yes	Yes	`sos`, `sos-hardening`	Implemented and verified
Hostel	Yes	Yes	`hostel`	Implemented and verified
Bus tracking	Yes	Yes	`transport` (11 tests); client `busGeo`, `myLocation`; Kotlin `BusMathTest`	Implemented and verified with synthetic GPS; a **real device or tracker was not verified**
Alumni network and mentorship	Yes	Yes	`alumni-flows`, `alumni-security`	Implemented and verified
iOS app	n/a	n/a	Swift tests exist but were **not run** (no Mac)	Unable to verify

Authentication and account security

Question	Answer
Purpose	Sign in, keep per-device sessions, lock out brute force, record security events
Roles	Everyone. Admins also see Security & Logs
Web entry	`client/src/pages/auth/Login.jsx`, `client/src/pages/settings/Security.jsx`, `client/src/pages/admin/SecurityLogs.jsx`, `client/src/components/layout/useIdleSession.js`
Kotlin	`screens/Login.kt`, `screens/Account.kt`, `core/Http.kt`, `core/SecureStore.kt`
Routes	`/api/auth/*` (login, refresh, logout, me, activity, sessions, change-password, login-history); `/api/admin/security/*`; `POST /api/users/:id/reset-password`
Logic	`authController.js`, `accountController.js`, `securityController.js`, `server/src/utils/tokens.js`, `loginSecurity.js`, `sessionIdle.js`, `securityLog.js`
Models	`User`, `Session`, `LoginHistory`, `LoginThrottle`, `SecurityLog`, `Activity`
Security rules	JWT pinned to HS256; access token bound to a live session; refresh rotation with replay detection; generic login error (anti-enumeration); IP throttling + account lockout; CSRF double submit for browsers; web 2-minute idle rule for non-exempt roles; email reset retired (410)
Debug first	`server/src/middleware/auth.js`, `middleware/security.js`, `client/src/services/api.js` (`baseQueryWithReauth`, `refreshSession`)
Gaps	Lockout email delivery depends on SMTP and is not tested end to end

Workflow: **web sign-in to an authenticated page**



Roles, users and role assignments

Question	Answer
Purpose	Create accounts, set class placement, assign Class In-Charge and HOD, suspend users
Roles	Admin (all users); faculty Class In-Charge (own class students only, `/class/students`)
Web entry	`client/src/pages/admin/AdminUsers.jsx`, `FacultyRoleAssignmentsTab.jsx`, `client/src/components/CreateUserModal.jsx`, `client/src/pages/class/ClassStudents.jsx`
Routes	`/api/admin/users*`, `/api/admin/role-assignments/*`, `/api/class-students*`
Logic	`adminController.js`, `adminRoleAssignmentController.js`, `userController.js`
Security	`pick()` whitelists fields; role and status changes bump `tokenVersion` and revoke sessions; Class In-Charge integrity is enforced in `User.js pre('validate')`
Tests	`roles`, `role-assignments`, `class-in-charge`, `class-in-charge-students`, `first-year-hod`, `college-wide-roles`, `user-title`

The "Science & Humanities" HOD owns **first year** for every branch. A branch HOD owns years 2 to 4 of their own department (`isFirstYearHod` / `isBranchHod` in `server/src/constants.js`). This rule appears in timetable, attendance and gate-pass scoping.

Dashboard and analytics

Question	Answer
Purpose	Role-specific home page; trends for students, classes, departments, the college, clubs and the gate
Web entry	`client/src/pages/Dashboard.jsx`, `client/src/pages/analytics/Insights.jsx`, `client/src/pages/admin/AdminAnalytics.jsx`, `client/src/components/charts.jsx` (Recharts)
Routes	`GET /api/dashboard`; `/api/analytics/{student,faculty,department,college,gate,club/:id}`; `GET /api/admin/analytics`
Logic	`dashboardController.js`, `analyticsController.js` (aggregation pipelines), `server/src/services/attendanceTotals.js` (warmed at start-up)
Gaps	Analytics numbers are not asserted by dedicated tests. Verify against small seeded data when you change a pipeline

Announcements, clubs, events and discussions

Module	Web pages	Routes	Controller / models	Notes
Announcements	`pages/announcements/`	`/api/announcements*`	`announcementController.js` / `Announcement`	Audience targeting (everyone, department, year, club), pinning; `announcement:changed` broadcast
Clubs	`pages/clubs/`	`/api/clubs*`	`clubController.js` / `Club`	Join request, then approve; per-club admins (`club.admins`); admin approves new clubs. **No direct tests**
Events	`pages/events/`	`/api/events*`	`eventController.js` / `Event`	Capacity + automatic waitlist, registration deadline, multi-day attendance, participants PDF (`services/eventParticipantsPdf.js`)
Discussions	`pages/discussions/`	`/api/discussions*`	`discussionController.js` / `Discussion`	Replies, upvotes, moderator lock/pin/hide; live replies via the `discussion:<id>` room. **No tests**

Module	Web pages	Routes	Controller / models	Notes
Reports and moderation	`pages/admin/AdminReports.jsx`	`/api/reports*`	`reportController.js`, `server/src/utils/moderation.js` / `Report`	Auto-flagging filter; actions dismiss/hide/warn/delete/suspend

`server/src/middleware/realtime.js` broadcasts `{entity, id, action}` after successful writes to these four modules, and every client refetches.

> **Exercise**
> Clubs and discussions have no tests. Write one test for `POST /api/clubs/:id/join` followed by approval (see Guide 07 for how). **Expected:** the test passes, and the student appears in `club.members`.

People directory, profiles and search

Question	Answer
Roles	`PEOPLE_DIRECTORY_ROLES`: students and club admins see **only their own class** (classmates + that class's faculty); faculty and HOD see scoped lists; college-wide roles see all
Web	`client/src/pages/people/People.jsx`, `Profile.jsx`, `client/src/pages/Search.jsx`
Routes	`GET /api/users`, `/api/users/people-filters`, `/api/users/:id`, `PUT /api/users/me`, `/api/users/export-pdf` (admin, principal), `GET /api/search`
Logic	`userController.js`, `server/src/utils/peopleScope.js`, `searchController.js` (MongoDB `$text` search)
Privacy	Email and phone are visible only to the owner, faculty and admins; `parentPhone` is never shown to classmates
Tests	`people`, `student-people`, `security-student-lists`, `profiles-export`

Achievements

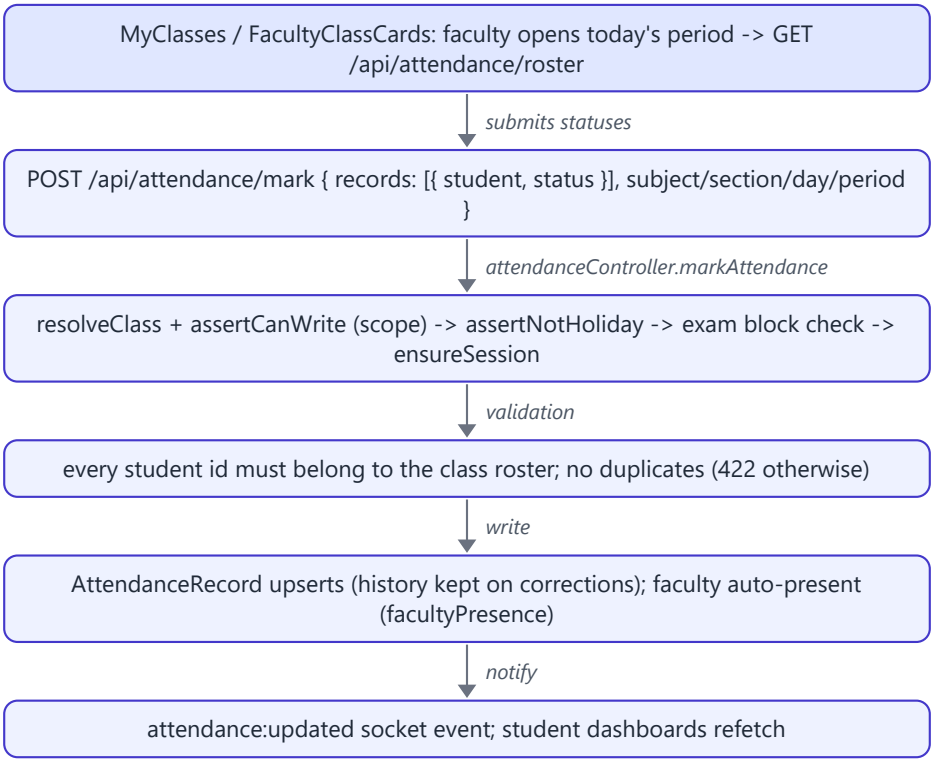
Principal and HOD post student achievements (the HOD only for their own department). Students can send "congratulations" and hide-requests, and an admin can restore soft-deleted posts. Web: `client/src/pages/achievements/`. Routes: `/api/achievements*`. Logic: `achievementController.js`. Model: `Achievement` (soft delete via `isDeleted`). Roles: `ACHIEVEMENT_POST_ROLES`, `ACHIEVEMENT_VIEW_ROLES` in `server/src/constants.js`. Test: `achievements.test.js`.

Attendance (students)

Question	Answer
Purpose	Faculty mark period attendance from the timetable; students see percentages, trends and history; corrections; reports; low-attendance monitor
Roles	Mark: faculty for their own scheduled class (HOD and admin within scope). Read college-wide (read-only): principal, dean, ao, chairman, coe, office
Web	`client/src/pages/attendance/` (`Attendance.jsx`, `MyClasses.jsx`, `OurClass.jsx`, `StudentAttendance.jsx`, `AttendanceReports.jsx`, `ODLeaveRequests.jsx`, ...), `client/src/pages/admin/AttendanceDashboard.jsx`, `client/src/services/reports.js`
Kotlin	`screens/Academics.kt`, `screens/AttendanceTools.kt`
Routes	`/api/attendance/*` (36 routes: roster, mark, my, trends, reports, corrections, od-leave, summary, faculty ...)
Logic	`attendanceController.js` (`getRoster`, `markAttendance`, ...), `odLeaveController.js`, `attendanceMonitorController.js`, `services/attendanceSessions.js`, `attendanceReport.js`, `attendanceReportPdf.js`, `attendanceTotals.js`, `attendanceLive.js`, `examAutoAttendance.js`, `facultyPresence.js`

Question	Answer
Models	`AttendanceSession`, `AttendanceRecord` (unique per student, subject, date, period), `AttendanceCorrectionRequest`, `ODLeaveRequest`
Rules	No marking on a holiday (`assertNotHoliday`); periods overlapping a published exam are blocked (409); approved OD counts specially; edits are allowed within a limited window (the README says 7 days); taking class attendance auto-marks the faculty member present for the day (`facultyPresence.js`)
Realtime	`attendance:updated`, `od_leave:updated`
Tests	`attendance-realtime`, `attendance-reports`, `attendance-timetable`, `attendance-od-leave`, `attendance-monitor`, `od-leave-scope`, `event-attendance-integration`

Workflow: **faculty marks a period**



Faculty attendance, early exit and period cover

HODs mark faculty attendance (`staffAttendanceController.js`, model `FacultyAttendance` with `source: 'auto' | 'manual'`). Faculty request early exits (`staffController.js`, model `FacultyEarlyExit`). HOD, principal and admin assign substitute cover for a period (model `PeriodCover`). Web: `client/src/pages/attendance/StaffAttendance.jsx`, `StaffMonitoring.jsx`. Routes: `/api/attendance/faculty/*`, `/api/staff/*`. Tests: `faculty-cover`, `faculty-class-cards`, `hod-dashboard-cards`.

Timetable, subjects, holidays and exams

Question	Answer
Roles	View: everyone except warden, security, alumni, office, transport (`TIMETABLE_BLOCKED_ROLES`). Write: admin (all) and HOD (own department; the first-year HOD owns year 1). Holidays: principal only. Exams: HOD (own scope) and COE (campus-wide)
Web	`client/src/pages/timetable/` (`Timetable.jsx`, `EditTimetable.jsx`, `MasterTimetable.jsx`, `FacultyAvailability.jsx`, `ExamTimetable.jsx`, `MySchedule.jsx`, `SlotModal.jsx`), `client/src/pages/attendance/Holidays.jsx`, `client/src/pages/admin/AdminAcademics.jsx`
Routes	`/api/timetable*`, `/api/subjects*`, `/api/holidays*`, `/api/exams*`
Logic	`timetableController.js` (clash checks; department scope never trusted from the client), `holidayController.js`, `examController.js` (`canManageExam`), `services/timetablePeriods.js` (breaks are not periods), `services/masterTimetablePdf.js`, `facultyAvailabilityPdf.js`, `services/exams.js`
Models	`TimetableSlot` (unique active slot per class, day and period), `Subject`, `Holiday`, `Exam`
Maintenance scripts	`npm run migrate:break-periods --prefix server` (dry run; add `-- --apply` to write), `npm run migrate:academic --prefix server`
Tests	`timetable-edit-periods`, `timetable-master`, `timetable-roles`, `faculty-availability`, `holidays`, `exam-timetable`, `academic`, `academic-scope`, `academic-authorization`

Study materials and the JNN Study Assistant (AI)

Question	Answer
Purpose	Faculty upload course material; students browse their class's material; an AI assistant answers questions using that material (plus optional web search)
Roles	Upload/edit: faculty (own teaching assignment), HOD (own department), admin. Blocked: warden, security, alumni, ao, coe, office, transport
Web	`client/src/pages/materials/StudyMaterials.jsx`, `UploadMaterialModal.jsx`, `client/src/components/StudyAssistant.jsx`, `client/src/services/studyAssistant.js`
Routes	`/api/study-materials*`, `/api/ai/study-assistant/*` (chat, conversations)
Logic	`studyMaterialController.js`, `server/src/utils/studyMaterialScope.js`, `server/src/services/ai/` (`studyAssistantService.js`, `groqService.js`, `pdfIndexer.js` using `unpdf`, `prompts.js`, `sources/`)
Models	`StudyMaterial`, `StudyMaterialChunk` (text-indexed PDF chunks), `AIConversation`, `AIUsageLog` (180-day TTL)
Security	Retrieval only from material the user may read; secrets redacted from output (`redactSecrets`); per-minute and per-hour AI rate limits; the assistant can only write the user's own conversation
External	Groq API (`GROQ_API_KEY`). Without a key, the assistant is disabled
Gaps	Answer quality and live Groq behaviour cannot be verified by unit tests

Workflow: `StudyAssistant.jsx` sends `POST /api/ai/study-assistant/chat` with `stream: true`. Then `aiController.chat` calls `askAssistant`, which loads the conversation, runs `gatherContext` (authorized chunks), builds the prompt and calls Groq. Answer pieces stream back as SSE `delta` events, followed by `done` with the saved message and its sources.

Chat

Question	Answer
Purpose	Private and group chat with live delivery, typing indicator, read receipts, pins, edit and delete, search
Roles	Private chat is limited by `server/src/utils/chatScope.js` (`privateChatAllowed`); groups need admin or principal approval (`/admin/chat-requests`)
Web	`client/src/pages/chat/Chat.jsx`, `GroupInfoModal.jsx`, `HodGroupModal.jsx`, `client/src/pages/admin/AdminChatRequests.jsx`
Routes	`/api/chat/*` (21 routes)
Logic	`chatController.js`, `server/src/services/chatService.js`; socket events in `server/src/config/socket.js` (`chat:join`, `chat:typing`, `chat:read`)
Models	`Conversation`, `Message`. Bodies are encrypted at rest with AES-256-GCM (`server/src/utils/cipher.js`)
Tests	`chat`, `chat-direct-group`, `chat-edit-message`, `chat-group-admin-scope`, `chat-groups`, `chat-scope`; plus `node server/scripts/realtime-smoke.mjs` against a running server

Notifications and push

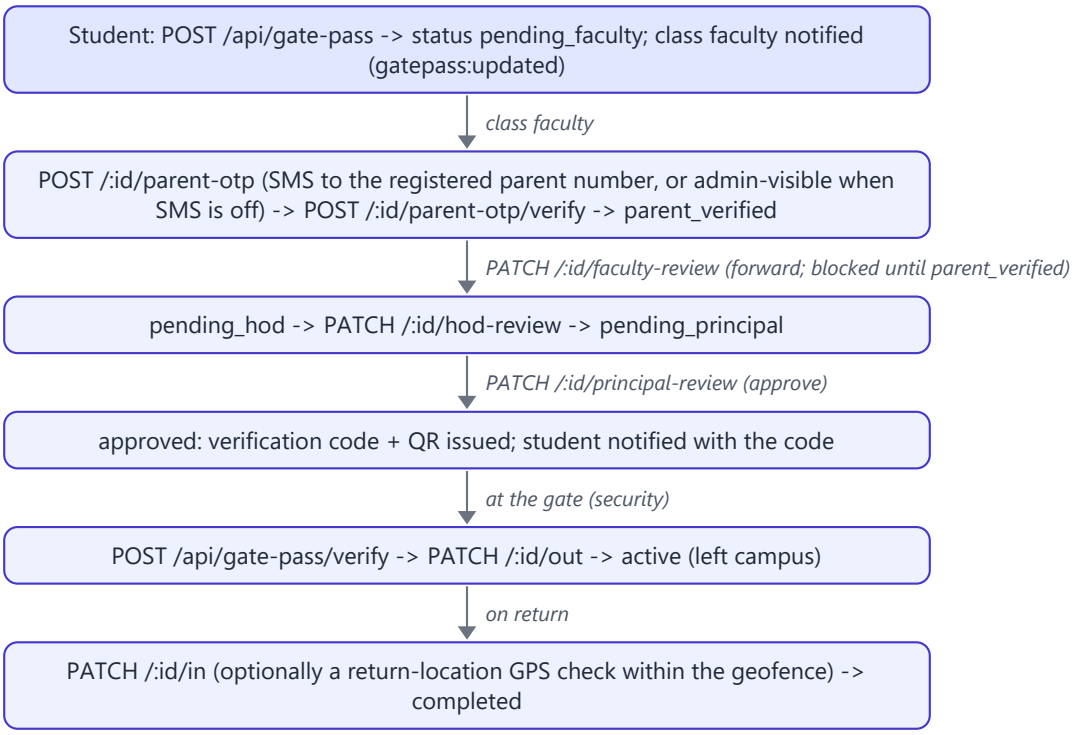
Notifications are created in exactly one place: `notifyUsers()` in `server/src/utils/notify.js`. It filters by user preference, stores the notification, emits it live to `user:<id>`, then calls `deliverPush()` in `server/src/services/pushService.js`. That function sends FCM pushes to the Kotlin app (`server/src/services/fcm.js`, model `DeviceToken`) and Expo pushes to the legacy app. Web: `client/src/pages/Notifications.jsx`, `client/src/components/layout/NotificationBell.jsx`. Preferences: `GET/PUT /api/notifications/preferences` (Kotlin `screens/NotificationSettings.kt`). Reminders and the daily digest come from `server/src/services/reminders.js`. Test: `notifications-push.test.js`. A real push to a phone was **not verified** in this audit.

Gate pass

Question	Answer
Purpose	Students request permission to leave campus. Staff approve in stages, security records exit and return, parents are verified by OTP
Variants	Normal pass (outing or home); day-scholar same-day outing; emergency pass (one chosen authority); bind groups / mass outpass (many students at once)
Roles	Student requests; class faculty, HOD and principal review; security verifies at the gate; admin and principal can revoke
Web	`client/src/pages/gatepass/` (`GatePass.jsx`, `GatePassDetail.jsx`, `GateVerify.jsx`, `ParentVerification.jsx`, `EmergencyGatePass.jsx`, `BindGroups.jsx`, `MassOutpassModal.jsx`, `ReturnStatus.jsx`), `client/src/components/QrScannerModal.jsx`, `client/src/utils/gateQr.js`, `returnLocation.js`
Kotlin	`screens/GatePass.kt`, `core/ReturnLocation.kt`, `ui/Native.kt` (QR scanner, location)
Routes	`/api/gate-pass/*` (36 routes)
Logic	`gatePassController.js`, `gatePassBindController.js`, `server/src/services/gatePassCodeQueue.js`, `gatePassReturn.js`, `server/src/services/otp/`
Models	`GatePass` (statuses in `GATE_PASS_STATUSES`), `GatePassBindGroup`, `OutpassOtp` (hashed and sealed OTP, 30-day TTL), `ReturnLocationCheck`
Security	The registered parent number from the database is always used (a client `parentPhone` is ignored); the QR holds an opaque code only; the OTP is hashed with limited attempts; one open pass per student (`gatePassLockUntil`); optimistic concurrency ("This pass was just reviewed by someone else")

Question	Answer
Background	Expiry sweep every 5 minutes (`expireGatePasses` in `server.js`); code-issuing queue for bulk approvals

Workflow: **normal gate pass**



Tests (more than 15 files): `campus`, `gatepass-bind`, `gatepass-return`, `gate-qr-scan`, `gate-exit-campus-day`, `gate-dashboard-roles`, `gate-card-popups-and-mass-outpass`, `emergency-gate-pass`, `day-scholar-outing*`, `outpass-otp`, `otp-*`, `parent-otp-alternate-mobile`, `principal-reject-approved`. **Not verified:** real SMS delivery through the college gateway.

Lost and found

Report lost or found items with a photo; the system suggests possible matches (never an ownership claim); staff link items and hand them over; owners close or withdraw their own reports. Web: `client/src/pages/lostfound/`. Kotlin: `screens/LostFound.kt`. Routes: `/api/lost-found*`. Logic: `lostFoundController.js`. Model: `LostFoundItem` (statuses `lost`, `found`, `possible_match`, `under_verification`, `returned`, `closed`). Privacy: contact details are visible only to the reporter and staff. Tests: 3 in `campus.test.js`.

Complaints (grievances)

Question	Answer
Purpose	Students raise grievances (optionally anonymous) that escalate through authorities if not resolved
Categories and targets	`COMPLAINT_CATEGORIES`, `COMPLAINT_ESCALATE_TO`, `COMPLAINT_ESCALATION_CHAINS` in `server/src/constants.js` (for example, hostel: warden, then ao, then chairman)
Roles	Create: students only. Handle: `COMPLAINT_AUTHORITY_ROLES` (faculty, hod, principal, dean, ao, warden) and admin. Dashboard: admin and chairman
Web	`client/src/pages/complaints/` (`Complaints.jsx`, `ComplaintDetail.jsx`, `ComplaintFormModal.jsx`), `client/src/pages/admin/ComplaintDashboard.jsx`
Routes	`/api/complaints*`

Question	Answer
Logic and models	`complaintController.js`; `Complaint`, `ComplaintHistory` (every status change recorded)
Rules	Ragging complaints get high priority automatically; "not resolved" becomes available after 8 hours (`COMPLAINT_NOT_RESOLVED_WAIT_MS`) and escalates to the next authority
Tests	`complaints.test.js`

Guide 02 traces the "Register grievance" button from the form to the database.

SOS alerts

Anyone signed in can raise an SOS (`POST /api/sos`). Responders (security, admin, hod, principal, chairman) get the full alert, including location. Every other role gets a reduced view. Burst limit: 5 per 10 minutes. Web: `client/src/components/sos/SOSModal.jsx`, `SOSBroadcastPopup.jsx`, `client/src/pages/sos/SOSAlerts.jsx`. Logic: `sosController.js`. Model: `SOSAlert`. Realtime: `sos:new`, `sos:updated`. Tests: `sos`, `sos-hardening`. Guide 03 walks through `createAlert` line by line.

Hostel

Read-only hostel summaries and student lists for `HOSTEL_VIEW_ROLES` (admin, principal, dean, ao, chairman, warden). Web: `client/src/pages/hostel/Hostel.jsx`. Routes: `GET /api/hostel/summary`, `GET /api/hostel/students`. Logic: `hostelController.js`. It uses `User.stayType` (`hosteler`, `day_scholar`). Test: `hostel.test.js`.

Bus tracking

Question	Answer
Purpose	A live map of college buses, routes, stops, progress and ETA; "My location" for the viewer
Roles	View: `TRANSPORT_VIEW_ROLES` (with `TRANSPORT_STUDENT_VISIBILITY=assigned`, students see only their assigned bus). Manage: admin and transport. Share a phone's location during a trip: `TRIP_SHARER_ROLES`
Web	`client/src/pages/transport/Transport.jsx`, `BusMap.jsx` (MapLibre GL), `TransportAdmin.jsx`, `busGeo.js`, `busMarker.js`, `myLocation.js`
Kotlin	`screens/BusTracking.kt`, `transport/BusMapController.kt`, `transport/TripLocationService.kt` (foreground service), `core/BusMath.kt`
Routes	`/api/transport/*` (config, buses, routes, ETA, search, me, trips, trip locations, assignments, tracker webhook)
Logic	`transportController.js`; `server/src/services/transport/` (`ingest.js` validates every fix, `access.js`, `eta.js`, `trips.js`, `events.js`, `geo.js`, `config.js`)
Models	`Bus` (latest location), `BusRoute` (stops, geometry), `BusTrip`, `BusLocation` (thinned history, TTL), `TransportAssignment`
Security and validity	Each GPS fix is checked for valid coordinates, age, future time, accuracy, speed, implied jump and rate; rejected fixes change nothing; the tracker webhook needs an HMAC signature; no background location permission on Android; the viewer's own location stays separate from bus positions
Dev tools	`node server/scripts/simulate-bus.mjs` sends **synthetic** positions (refuses to run in production)
Tests	`server/tests/transport.test.js` (11), `client/tests/busGeo.test.mjs`, `client/tests/myLocation.test.mjs`, Kotlin `BusMathTest.kt`
Not verified	Real vehicle trackers; real-device GPS accuracy

Workflow: a staff phone shares its location. `POST /api/transport/trips` starts a trip, and the Kotlin `TripLocationService` sends fixes to `POST /api/transport/trips/:id/locations`. `ingest.js` validates them, updates `Bus.lastLocation`, writes thinned `BusLocation` history and emits `bus:location` to `bus:<id>` and `transport:all`. `Transport.jsx` moves the marker.

Alumni network and mentorship

Alumni profiles and verification, a jobs board with applications, alumni events with RSVP and check-in, chapters with posts and comments, CSV import with email invites, mentorship requests, slots and sessions, and analytics. Web: `client/src/pages/alumni/` (13 tab components). Routes: `server/src/routes/alumniRoutes.js` (78 routes, mounted at `/api/alumni`, `/api/mentorship`, `/api/mentorship-requests`, `/api/auth/alumni-claim`). Logic: `server/src/controllers/alumni/`, `server/src/utils/alumniPrivacy.js`, `alumniSchedulers.js`. Models: the `Alumni*`, `Chapter*`, `Mentorship*` and `JobApplication` files. Tests: `alumni-flows`, `alumni-security`. Seeds: `npm run seed:alumni --prefix server`.

Android: the Kotlin app

Question	Answer
Location	`mobile-kotlin/` (package `com.vexon.campus`; the debug build installs as `com.vexon.campus.native`)
Structure	`core/` (Http, Api, Query cache, Realtime, SecureStore, Config, Format, BusMath, ReturnLocation), `nav/Nav.kt`, `screens/` (one file per area), `transport/`, `push/Push.kt`, `ui/`
Screens (routes in `screens/Routes.kt`)	announcements, clubs, events, discussions, search, calendar, study-materials, settings, admin create-user and chat-requests, lost-found, chat, timetable, attendance, od-leave, hostel, class students, sos, gate-pass (new, in, out, mass), complaints, achievements, holidays, exams, transport, people, alumni
Permissions (manifest)	INTERNET, POST_NOTIFICATIONS, COARSE/FINE location, CAMERA, FOREGROUND_SERVICE(_LOCATION); **background location and RECORD_AUDIO are explicitly removed**
Build	`cd mobile-kotlin`, then `gradlew assembleDebug`; release with `gradlew assembleRelease` (signed with the debug key until a real keystore is configured)
Backend URL	`vexon.apiUrl` in `mobile-kotlin/gradle.properties` is currently the deployed Render URL, so **debug builds also talk to the deployed server** unless you build with `-Pvexon.apiUrl=http://10.0.2.2:5000` (the emulator's view of your PC). The `10.0.2.2` default applies only when the property is empty. Users can also change the server on the login screen
Tests	8 unit-test files, 50 tests, which passed in this audit; no instrumented (device) tests were run

Android: the Expo app (older)

`mobile/` is the React Native / Expo SDK 57 client that the Kotlin app ports. It uses expo-router (`mobile/src/app`), RTK Query (`mobile/src/services`) and EAS builds (`mobile/eas.json`). Its 13 unit tests passed in this audit. Treat it as a **legacy client**: the server still supports its Expo push tokens (`POST /api/auth/push-token`).

iOS app

Evidence status

Unable to verify. `mobile-ios/` contains a SwiftUI app (`mobile-ios/VEXON/`) and a platform-independent Swift package (`mobile-ios/Packages/VEXONKit/`) with Swift Testing suites. Its README states that it has never been compiled. The folder is untracked in git, so commit it deliberately (or add it to `.gitignore`) once you decide. Features present as source: Auth, Dashboard, Attendance, Timetable (with exams), Complaints, Chat, Notifications, Profile, Bus Tracking. A Mac with Xcode 16+ is required (`mobile-ios/Scripts/generate-project.sh`).

Practice: explain one workflow end to end

Exercise

Pick **Complaints** or **SOS**. Using only this guide and the files it names, write ten lines that describe the flow from the button the user taps to the MongoDB document and the notification the receiver sees. **Expected:** your description names a web component, an RTK hook, a route with its middleware, a controller function, a model and a socket event, each with a correct path.

Source references

`server/src/routes/index.js` · `server/src/routes/alumniRoutes.js` · `server/src/constants.js` · `server/src/controllers/` (all) · `server/src/services/` (transport, ai, otp, attendance) · `server/src/models/` · `server/tests/` · `client/src/App.jsx` · `client/src/pages/` · `client/src/services/api.js` · `mobile-kotlin/app/src/main/AndroidManifest.xml` · `mobile-kotlin/app/src/main/java/com/vexon/campus/screens/Routes.kt` · `mobile-kotlin/README.md` · `mobile-ios/README.md` · `mobile/package.json`